

Машинное обучение 1

Проверочная работа №1

Ключ и пояснения

Возможные ответы к трём задачам, всего 10 баллов. Допустимы другие обозначения и равносильные формулы с пояснениями. В пункте 3.2 требовались две модификации. Все разобранные в курсе варианты собраны ниже для повторения.

Задача 1. Линейная регрессия и регуляризация

1.1. Предсказание и MSE (2 балла)

Для объекта $x = (x_1, \dots, x_d)$ линейная модель даёт прогноз

$$a_w(x) = w_0 + \sum_{j=1}^d w_j x_j.$$

Здесь d обозначает число признаков, w_j является весом признака x_j , а w_0 задаёт свободный член. По выборке из ℓ пар (x_i, y_i) выбирают веса $w = (w_0, \dots, w_d)^\top$, минимизирующие MSE:

$$Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} (a_w(x_i) - y_i)^2 \longrightarrow \min_w.$$

x_i обозначает вектор признаков i -го объекта, y_i является его истинным ответом. Если A содержит признаки и столбец единиц, то $\hat{y} = Aw$ и $Q(w) = \|Aw - y\|_2^2 / \ell$.

1.2. Ridge и Lasso (2 балла)

Регуляризация ограничивает величину весов и помогает уменьшить переобучение. При сильно связанных признаках она также помогает сделать оценки весов устойчивее. К ошибке добавляют штраф:

$$J_\lambda(w) = Q(w) + \lambda R(w), \quad \lambda \geq 0.$$

$$R_{\text{Ridge}}(w) = \sum_{j=1}^d w_j^2, \quad R_{\text{Lasso}}(w) = \sum_{j=1}^d |w_j|.$$

Ridge сжимает веса к нулю, обычно не получая разреженного решения. Lasso может сделать часть весов точно нулевыми и тем самым исключить признаки из прогноза.

Свободный член w_0 в этих формулах не штрафуются. Перед регуляризацией признаки приводят к сопоставимому масштабу. Чем больше λ , тем сильнее штраф относительно ошибки. Его выбирают по валидации.

Задача 2. Линейная классификация

2.1. Отступ (2 балла)

Пусть $y_i \in \{-1, +1\}$. Линейный скор и отступ имеют вид

$$f_w(x) = w_0 + \sum_{j=1}^d w_j x_j, \quad M_i = y_i f_w(x_i).$$

Класс определяют по знаку сора. Поэтому

- $M_i > 0$: знак сора совпадает с меткой, ответ верный.
- $M_i < 0$: знаки различаются, ответ неверный.
- $M_i = 0$: объект лежит на границе решений.

В лекции принято считать нулевой отступ ошибкой, поэтому используется индикатор $[M_i \leq 0]$. Для ответа на этот пункт достаточно указать, что нулевой отступ означает границу. Сам отступ без нормировки не является расстоянием до неё.

2.2. Доля ошибок и гладкая потеря (1 балл)

Лекционный функционал

$$Q_{0/1}(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} [M_i \leq 0]$$

кусочно постоянен по весам. Пока объекты не пересекли границу, небольшое изменение весов не меняет число ошибок. Градиент почти всюду равен нулю, а в точках скачка производной нет. Обычный градиентный спуск не получает полезного направления.

Вместо индикатора можно взять **логистическую потерю**:

$$L(M) = \ln(1 + e^{-M}).$$

Она гладкая и убывает при росте отступа. Обучение минимизирует среднюю потерю по объектам. Из лекции также подходят экспоненциальная и сигмоидальная потери:

$$L_{\text{exp}}(M) = e^{-M}, \quad L_{\text{sigmoid}}(M) = \frac{2}{1 + e^M}.$$

Достаточно было одной формулы. Hinge-потеря $\max(0, 1 - M)$ не гладкая при $M = 1$, поэтому не отвечает требованию этого пункта.

Задача 3. Градиентный спуск

3.1. Идея метода (1 балл)

Градиентный спуск численно минимизирует функцию, например ошибку модели по весам. Градиент направлен в сторону наибольшего локального роста, поэтому веса меняют в противоположную сторону:

$$w^{(k)} = w^{(k-1)} - \eta_k \nabla J(w^{(k-1)}), \quad \eta_k > 0.$$

Здесь k обозначает номер шага, η_k задаёт learning rate, а J может быть ошибкой Q или ошибкой со штрафом. При достаточно малом шаге и ненулевом градиенте гладкая функция уменьшается. Слишком большой шаг может привести к расходимости.

3.2. Модификации градиентного спуска (2 балла)

Для полного ответа достаточно назвать две разные модификации и кратко объяснить каждую. Например: «Momentum учитывает прошлое направление движения. AdaGrad меняет масштаб шага для каждого веса по накопленной сумме квадратов градиентов». Формулы здесь не требовались.

В расширенном разборе сначала различаем способы вычислять направление, затем способы строить по нему шаг. Их можно сочетать, например использовать Adam с мини-батчами. Обозначим потерю на объекте через $q_i(w)$:

$$Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} q_i(w), \quad w^{(k)} = w^{(k-1)} - \eta_k g_k.$$

Полный GD: вся выборка

На каждом шаге пересчитывают градиенты всех объектов:

$$g_k = \frac{1}{\ell} \sum_{i=1}^{\ell} \nabla q_i(w^{(k-1)}).$$

Получают точный градиент Q , но один шаг на большой выборке дорог. Это базовый метод для сравнения. Повтор его описания не заменяет вторую модификацию в ответе.

SGD: один случайный объект

Случайно и равновероятно выбирают объект i_k и вычисляют

$$g_k = \nabla q_{i_k}(w^{(k-1)}).$$

Шаг дешевле, но направление шумное. Его математическое ожидание равно полному градиенту. Ошибка на всей выборке не обязана уменьшаться после каждого обновления.

Mini-batch GD: небольшая порция

Выбирают порцию B_k из b объектов и усредняют её градиенты:

$$g_k = \frac{1}{b} \sum_{i \in B_k} \nabla q_i(w^{(k-1)}).$$

Усреднение уменьшает шум оценки по сравнению с одним объектом, а вычисления удобно выполнять параллельно. При $b = 1$ получаем SGD, при $b = \ell$ и использовании всей выборки получаем полный GD.

SAG: память о градиентах объектов

Для каждого объекта хранят последний вычисленный градиент z_i . Сначала все записи равны нулю. На шаге выбирают объект i_k и обновляют только его запись:

$$z_{i_k} \leftarrow \nabla q_{i_k}(w^{(k-1)}), \quad g_k = \frac{1}{\ell} \sum_{i=1}^{\ell} z_i.$$

Используется информация о разных объектах при вычислении одного нового градиента за шаг. Остальные записи могли быть получены при прежних весах, поэтому их среднее обычно не равно текущему полному градиенту. Нужна память для записей.

3.2. Инерция и адаптивный масштаб шага

В формулах ниже g_k обозначает текущий градиент или его оценку по мини-батчу. Индекс j соответствует отдельному весу. Все накопители перед первым шагом равны нулю.

Momentum: учитываем прошлый шаг

Лекционная запись хранит сам вектор шага h_k :

$$h_k = \alpha h_{k-1} + \eta g_k, \quad w^{(k)} = w^{(k-1)} - h_k, \quad 0 \leq \alpha < 1.$$

Если направления согласованы, движение усиливается. При колебаниях часть вкладов компенсируется. Это помогает двигаться по вытянутой поверхности ошибки, но параметры шага и инерции всё равно нужно выбирать.

В семинаре 4 при постоянном η используется равносильная запись через скорость $v_k = \alpha v_{k-1} + g_k$, затем $w^{(k)} = w^{(k-1)} - \eta v_k$. В этом случае $h_k = \eta v_k$.

AdaGrad: накапливаем квадраты градиентов

Каждый вес получает свой масштаб шага:

$$G_{k,j} = G_{k-1,j} + g_{k,j}^2, \quad w_j^{(k)} = w_j^{(k-1)} - \frac{\eta g_{k,j}}{\sqrt{G_{k,j} + \varepsilon}}.$$

При том же текущем градиенте большая накопленная сумма означает меньшее обновление. Для редко активных признаков масштаб уменьшается медленнее. Сумма $G_{k,j}$ не убывает, поэтому со временем шаги могут стать слишком маленькими.

Например, при $\eta = 1$, $g_{k,j} = 1$ и малом ε накопленные суммы 5 и 2 дают изменения примерно $-0,45$ и $-0,71$. Это пример из лекции 4.

RMSprop: постепенно забываем старые квадраты

Вместо суммы всех квадратов используют скользящее среднее:

$$G_{k,j} = \beta G_{k-1,j} + (1 - \beta) g_{k,j}^2, \quad 0 \leq \beta < 1,$$
$$w_j^{(k)} = w_j^{(k-1)} - \frac{\eta g_{k,j}}{\sqrt{G_{k,j} + \varepsilon}}.$$

Старые большие градиенты постепенно теряют влияние. В отличие от AdaGrad, $G_{k,j}$ может уменьшаться, а эффективный шаг при том же градиенте может снова вырасти. При $\beta = 0,9$ в новом среднем остаётся 90% прошлого значения и добавляется 10% текущего квадрата.

В формулах AdaGrad и RMSprop выше $\varepsilon > 0$ стоит под корнем, как в лекции. В коде семинара 4 используется распространённый вариант $\sqrt{G_{k,j} + \varepsilon}$. Оба варианта защищают от деления на ноль, но численно не тождественны.

3.2. Adam и выбор длины шага

Adam: усредняем направление и квадраты

Adam сочетает идеи Momentum и RMSprop. При $m_0 = v_0 = 0$ и $k = 1, 2, \dots$ вычисляют

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k, \quad v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k^2.$$

Здесь $0 \leq \beta_1, \beta_2 < 1$. Квадрат и остальные операции с векторами выполняются по координатам. Поправки на нулевые начальные значения имеют вид

$$m_k^c = \frac{m_k}{1 - \beta_1^k}, \quad v_k^c = \frac{v_k}{1 - \beta_2^k}.$$

Обновление весов:

$$w^{(k)} = w^{(k-1)} - \eta \frac{m_k^c}{\sqrt{v_k^c + \varepsilon}}, \quad \varepsilon > 0.$$

Среднее m_k сглаживает направление, а v_k задаёт масштаб для каждого веса. Именно эти две идеи достаточно было объяснить на проверочной. Поправки выписаны здесь для связи с реализацией из семинара 4.

Как выбирают длину шага

В лекциях рассматривались три подхода:

- **Постоянный шаг.** Одно значение η на всех итерациях.
- **Расписание.** Шаг уменьшают в заданные моменты или по формуле, например $\eta_k = \eta_0 / (1 + ck)$ при $c > 0$.
- **По изменению ошибки.** В полном GD пробуют шаг и проверяют целевую функцию. Если она выросла, уменьшают пробный шаг и повторяют попытку.

Для убывающего шага в SGD в лекции приводились условия

$$\sum_{k=1}^{\infty} \eta_k = \infty, \quad \sum_{k=1}^{\infty} \eta_k^2 < \infty.$$

Например, им удовлетворяют $\eta_k = 0,1/k$ и $\eta_k = 0,1/k^{0,75}$. Эти условия входят в результаты о сходимости при дополнительных предположениях о функции и шуме. Сами по себе они не гарантируют сходимость в произвольной задаче.

Где это было в материалах курса

- Лекция 2, слайды 5, 7, 14–17: линейная модель и MSE.
- Лекция 3, слайды 14–23, 25–40: регуляризация, GD, SGD и mini-batch.
- Лекция 4, слайды 4–23, 26–39: SAG, шаг, адаптивные методы, отступ и потери.
- Семинар 3 и семинар 4: вычислительные примеры и реализации методов.